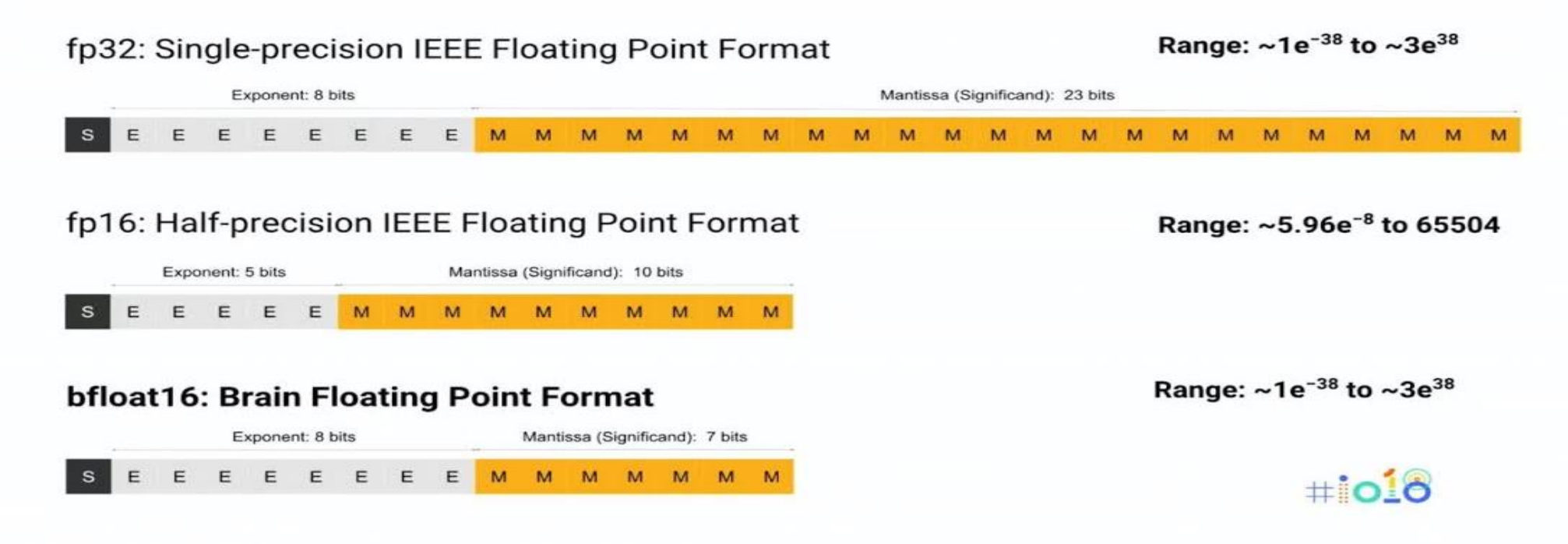
Floating-point formats used/proposed by various companies

Sparsh Mittal

Bfloat16


fp32: Single-precision IEEE Floating Point Format
Range: ~1e-38 to ~3e38
Exponent: 8 bits
Mantissa (Significand): 23 bits
S E E E E E E E E M M M M M M M M M M M M M M M M M M M M M M M
fp16: Half-precision IEEE Floating Point Format
Range: ~5.96e-8 to 65504
Exponent: 5 bits
Mantissa (Significand): 10 bits
S E E E E E M M M M M M M M M M
bfloat16: Brain Floating Point Format
Range: ~1e-38 to ~3e38
Exponent: 8 bits
Mantissa (Significand): 7 bits
S E E E E E E E E M M M M M M M

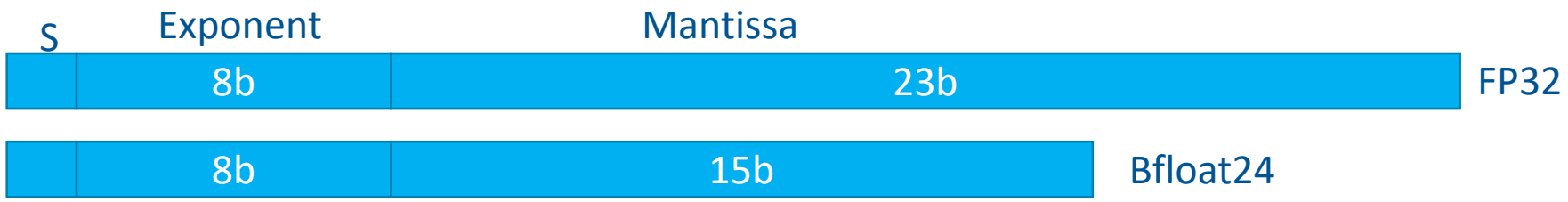
Advantages of Bfloat16

- Bfloat16 works as well as FP32 with better performance and lower memory usage.
- Multiplier size scales with square of the mantissa width.
  - With fewer mantissa bits than FP16, bfloat16 multipliers are ~1/2 size of a typical FP16 multiplier, and 1/8X of FP32 multiplier!
- BFloat16 has same exponent size as FP32 => identical behavior for underflows, overflows, and NaNs.
- However, bfloat16 handles denormals differently from FP32: it flushes them to zero.
  - FP16 requires special handling of denormals
- BFloat16 is almost drop-in replacement for FP32 for training and running DNNs

Limitation of BFloat16

- Having just 7 fraction bits limits its usefulness in computations where a large number of terms need to be added together, for example, dot-product.
- To overcome this, accumulation with bfloat16 is usually performed using a special instruction where the result is a FP32 number.
- Thus, floating-point units (FPUs) designed for bfloats are hybrid FP16 and FP32 FPUs, with logic savings mostly restricted to mantissa multiplication vis-à-vis true FP32 FPUs.

Bfloat24



Lower area overhead than FP32

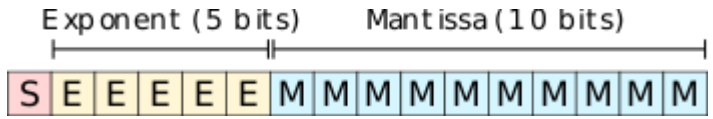
Supported in Intel's Stratix 10 NX FPGA

https://dl.acm.org/doi/pdf/10.1145/3520197

Microsoft MS-FP8, used in Brainwave

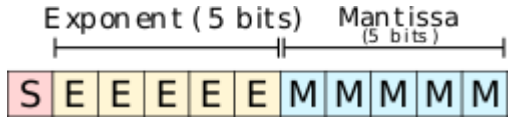
IEEE 754 half-precision floating point

FP16

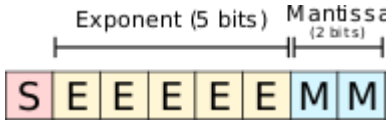


Microsoft's proprietary parameterizable narrow precision format wrapped in FP16 interfaces

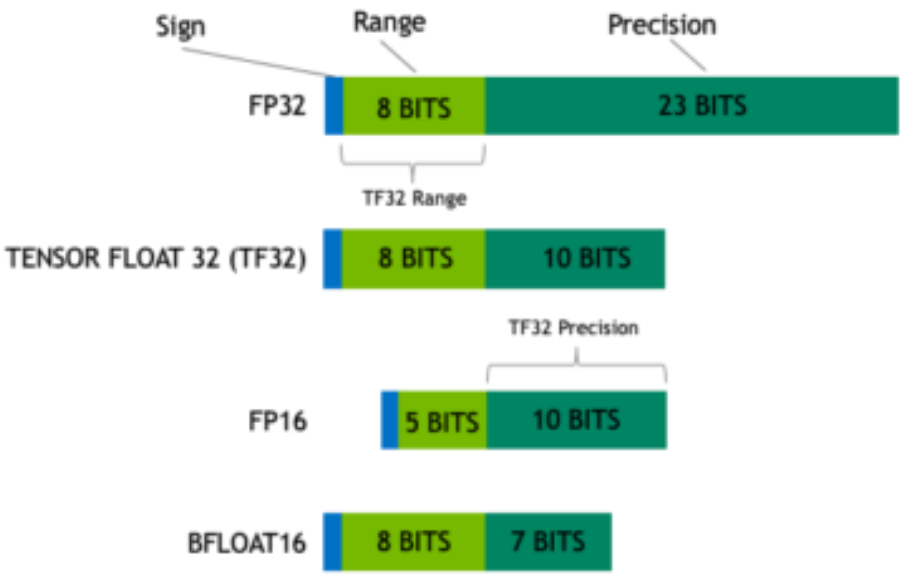
MSFP11



MSFP8



NVIDIA TF32 [sometimes called FP19]



TF32 strikes a balance that delivers performance with range and accuracy

- TF32 uses same 10-bit mantissa as FP16, which has more than sufficient margin for the precision requirements of AI workloads.
- TF32 uses same 8-bit exponent as FP32 so it can support the same numeric range.

https://blogs.nvidia.com/blog/2020/05/14/tensorfloat-32-precision-format/

Comparison

Limitations of FP16:

Either training did not converge or required extra efforts to stabilize due to exploding and vanishing gradients, outlier weights, etc.

Bfloat16:

Better than FP16, however, some models may not converge with Bfloat16.

TensorFloat-32 (FP-19)

It allows noticeably faster arithmetic than fp32,

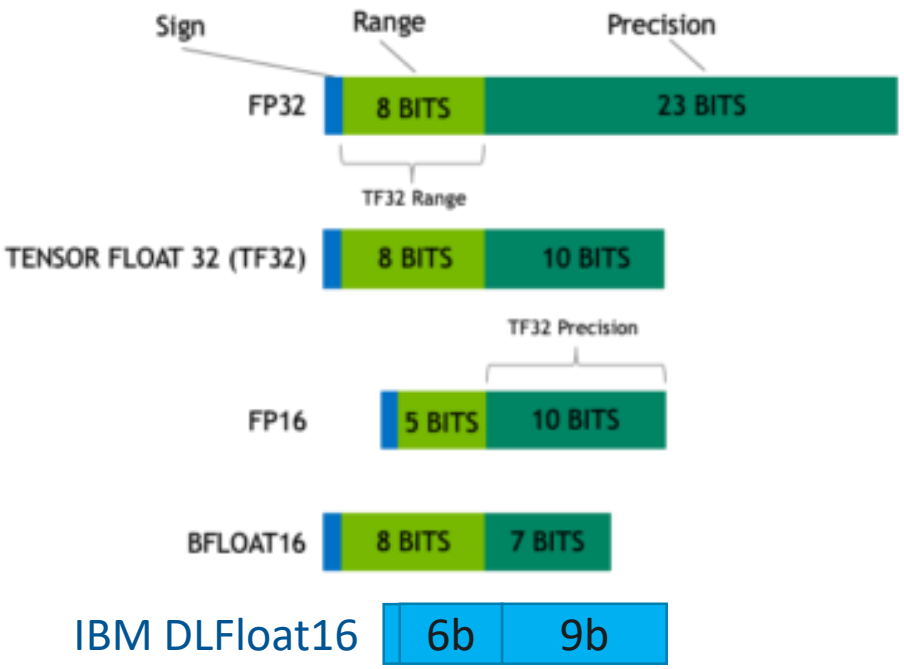
Maintains enough numeric stability for the models to reliably converge without extra programming effort.

So far, it appears to be sweet spot for AI

NVIDIA’s Blackwell, Hopper and Turing Tensor Cores enable operations in FP8, FP4, and 1-bit, respectively.

IBM’s DLFloat-16

Definition



6b for exponent, 9b for mantissa

Value is computed as:

$$X = (-1)^s * 2^{(e-b)} * (1 + \frac{m}{512})$$

Agrawal, Ankur, et al. "Dlfloat: A 16-b floating point format designed for deep learning training and inference." *IEEE ARITH*, 2019.

No subnormal (denormal) numbers

It has no subnormal numbers
With 6b exponents, this format has sufficient dynamic range even without subnormals.

Eliminating subnormals simplifies FPU logic

Hardware support for subnormal numbers requires a wider data flow and adds to the logic depth of a conventional fused-multiply-add FPU.

Simplified NaN, Infinity and Zero

- For 1 sign-bit and F fraction bits, IEEE-754 has $2*(2^F-1)$ representations for NaN and 2 representations for Infinity
- DLFloat16 designates the last symbol (e=63, m=511) to represent Nan-Infinity.
- The rest of the last binade is used for normal numbers.
- Not having to distinguish between multiple flavors of NaN and infinity simplifies FPU logic.
- They ignore sign bits when an input or output is zero or Nan-Infinity.
- Only one rounding-mode: round-nearest up. Simplifies FPU.

Results

Theirs is a true FP16 FPU that can be contrasted against mixed-precision (FP16-FP32) FPUs.

Training of LSTM using DLFloat achieves FP32 performance

DLFloat16 FPU is >20X smaller than our FP64 FPU

FP8

Already implemented in products (e.g. NVIDIA Hopper)

Two formats: E4M3 (4 exponent, 3 mantissa) and E5M2

E4M3 for weight and activation tensors

E5M2 for gradient tensors.

https://arxiv.org/pdf/2209.05433.pdf

Two FP8 Formats

1. (sign, exponent, mantissa) bits of (1, 4, 3)
    1. Lower dynamic range
2. (sign, exponent, mantissa) bits of (1, 5, 2)
    1. Higher dynamic range

Results with FP8

Table 2: Image Classification Models, ILSVRC12 Validation Top-1 Accuracy

Model	Baseline	FP8
VGG-16	71.27	71.11
VGG-16 BN	73.95	73.69
Inception v3	77.23	77.06
DenseNet 121	75.59	75.33
DenseNet 169	76.97	76.83
Resnet18	70.58	70.12
Resnet34	73.84	73.72
Resnet50 v1.5	76.71	76.76
Resnet101 v1.5	77.51	77.48
ResNeXt50	77.68	77.62
Xception	79.46	79.17
MobileNet v2	71.65	71.04
DeiT small	80.08	80.02

Table 4: NLP Models, Perplexity

Model	Baseline	FP8
Transformer-XL Base	22.98	22.99
Transformer-XL Large	17.80	17.75
GPT 126M	19.14	19.24
GPT 1.3B	10.62	10.66
GPT 5B	8.94	8.98
GPT 22B	7.21	7.24
GPT 175B	6.65	6.68